UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Software

Asignatura: Aplicaciones Distribuidas

TA-IND-01 — Informe Académico Individual

Análisis crítico de RabbitMQ como sistema distribuido de mensajería y comparación con Apache Kafka

Estudiante: Luis Mario Zambrano Párraga

Paralelo: **A**

Fecha: 13 de noviembre de 2026

Repositorio público del grupo GHLZ:

`https://github.com/Yukisan-uwo/TA-IND-01-Informe-Academico-Individual-Analisis-Critico-de-Sistemas-Distribuidos-Grupo`

Directorio del estudiante: `TA-IND-01-Zambrano Parraga Luis Mario`

Resumen

RabbitMQ permite intercambiar mensajes entre aplicaciones sin exigir que productor y consumidor trabajen simultáneamente. En este informe analizo su arquitectura distribuida y la comparo con Apache Kafka mediante publicaciones científicas recientes y pruebas locales. El experimento se realizó con RabbitMQ 4.2.9, tres contenedores Docker y una cola de quórum de tres miembros. Se examinan las transparencias de distribución, los modelos de comunicación, CAP, PACELC y la tolerancia a fallos. Con Python y Pika se enviaron mensajes con confirmaciones del broker y se recuperaron con confirmaciones manuales. Al detener el líder inicial, otro nodo asumió su función y fue posible consumir cinco mensajes que habían quedado pendientes. Cuando solo permaneció un miembro disponible, la cola perdió su líder operativo; al restablecer la mayoría, los mensajes continuaron disponibles. Los resultados muestran una ventaja de la replicación por consenso, pero también su dependencia del quórum y del punto de conexión del cliente. La comparación con Kafka permite discutir situaciones de uso diferentes, sin afirmar que un sistema sea mejor en todos los casos. No se midieron latencias ni se simularon particiones de red.

Palabras clave: RabbitMQ, Raft, cola de quórum, replicación, tolerancia a fallos, Kafka.

1. Introducción

En los sistemas distribuidos es común que una aplicación genere información más rápido de lo que otra puede procesarla. También pueden existir interrupciones en algún componente sin que necesariamente deba detenerse todo el servicio. Una forma de afrontar el intercambio de información consiste en utilizar un intermediario de mensajes. Maharjan et al. explican que este mecanismo desacopla a los productores de los consumidores y permite que las aplicaciones funcionen de manera independiente [1].

El sistema elegido fue RabbitMQ, un broker de mensajería de código abierto utilizado para coordinar tareas y transferir mensajes entre servicios. Su arquitectura integra clientes, intercambios, colas y consumidores. RabbitMQ también puede formar clústeres y utilizar colas replicadas mediante consenso Raft [2]. Apache Kafka se seleccionó como comparación porque también resuelve la distribución de datos entre aplicaciones, aunque organiza la información como registros de eventos en temas y particiones. La comparación es pertinente, pero hay diferencias que impiden tratarlos como productos idénticos [1].

Se escogió RabbitMQ porque fue posible ejecutarlo localmente, consultar sus nodos y provocar fallos controlados. En el laboratorio se utilizaron tres contenedores sobre una misma computadora; esta condición permitió observar la caída de procesos, pero no una caída de servidores físicos independientes. El análisis se organiza en dos partes complementarias: revisión de los conceptos de la Unidad 1 y discusión de resultados obtenidos con comandos, capturas y programas Python guardados en el repositorio. No se presume que los resultados de este laboratorio sean extrapolables a un despliegue empresarial.

2. Arquitectura de RabbitMQ

2.1. Componentes y comunicación

El recorrido de un mensaje empieza en el productor, que publica datos hacia un *exchange*. El exchange decide a qué cola enviarlos utilizando reglas de enlace o *bindings*. Finalmente, un consumidor obtiene los mensajes de la cola. Esta separación entre el envío, el enrutamiento y el consumo aparece explicada en el artículo de Maharjan et al. [1].

En el código del experimento utilicé el exchange predeterminado, expresado con `exchange=""`. La clave de enrutamiento fue el nombre `taind01_mensajes`; de este modo el mensaje llegaba a la cola creada previamente. El productor se ejecutó desde Python 3.11.9 con Pika 1.4.4, conectado por AMQP 0-9-1 sobre TCP al puerto 5672. El consumidor usó inicialmente la misma dirección, pero se preparó para conectarse también al puerto 5673 cuando fue necesario. Esta configuración puede verificarse en `productor.py`, `consumidor.py` y en los registros del experimento.

2.2. Clúster y cola de quórum

La configuración incluyó tres brokers llamados `rabbit1`, `rabbit2` y `rabbit3`. Una cola de quórum no equivale a tres colas independientes: es una cola lógica con un registro replicado cuyos miembros siguen un protocolo de consenso. Uno actúa como líder y los demás como seguidores. Según la documentación oficial, las colas de quórum de RabbitMQ utilizan Raft, requieren una mayoría operativa y pueden elegir un nuevo líder cuando falla el anterior [2].

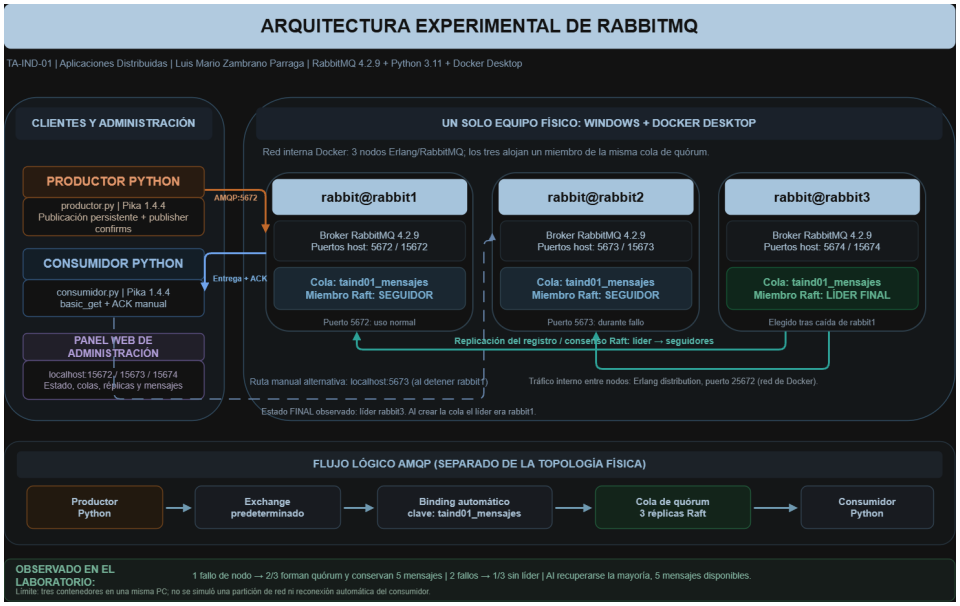


Figura 1: Arquitectura del laboratorio con los tres brokers, la cola de quórum y los clientes Python; diagrama editable elaborado en draw.io a partir del bosquejo manuscrito.

La estructura del laboratorio comprende el productor Python, el exchange predeterminado, la cola `taind01_mensajes`, el consumidor, los tres nodos y las conexiones AMQP y de replicación. El

boceto a mano se conserva en el Anexo A. Para interpretar la arquitectura es necesario distinguir el recorrido lógico de los mensajes y la replicación interna del registro Raft, que son mecanismos relacionados, pero no idénticos.

3. Análisis de los conceptos de la Unidad 1

3.1. Transparencias de distribución

La transparencia consiste en ocultar a las aplicaciones ciertas diferencias entre utilizar un recurso local y uno distribuido. Chalabi y Slimani revisan transparencia de acceso, ubicación, concurrencia, fallos, replicación y migración en sistemas de almacenamiento [3]. Para la reubicación durante el uso se toma como referencia adicional la continuidad estudiada en la migración en vivo, sin confundirla con el simple cambio de líder de una cola [4]. Ninguno de esos artículos demuestra por sí solo que RabbitMQ implemente todas las transparencias; por ello se distinguen las funciones documentadas de lo observado en nuestras pruebas.

Tabla 1: Evaluación de las siete transparencias en el entorno estudiado.

Transparencia	Grado	Evidencia y limitación
Acceso	Parcial	Pika ofrece operaciones de publicar/consumir, pero hay que configurar host, puerto y credenciales.
Ubicación	Parcial	La cola se consultó desde un broker diferente de su líder; el cliente todavía conoce un endpoint concreto.
Migración	No probada	No se trasladó una cola ni un proceso entre máquinas.
Reubicación	No probada	La elección de otro líder no equivale a mover una sesión activa sin interrupción.
Replicación	Parcial	El cliente trabaja con una cola lógica, mientras la administración muestra los tres miembros Raft.
Concurrencia	No probada	La secuencia de cinco mensajes no permite concluir qué ocurre con productores concurrentes.
Fallos	Parcial	Los mensajes siguieron disponibles con dos miembros, pero el consumidor cambió manualmente de puerto.

En particular, la falla de `rabbit1` no fue completamente transparente para nuestro consumidor: el programa inicialmente conectaba a `127.0.0.1:5672` y se modificó la variable `RABBITMQ_PORT` a `5673`. Esto corresponde a una limitación de nuestra configuración de cliente, no a la afirmación de que RabbitMQ nunca permite reconexión automática. La guía oficial advierte que, cuando se pierde una conexión, el cliente debe recuperarla o establecer otra, aunque algunas bibliotecas incorporan mecanismos para hacerlo [5].

3.2. CAP y PACELC

CAP estudia el compromiso entre consistencia y disponibilidad cuando existe una partición de red; no afirma que en cualquier condición el sistema deba elegir solamente dos propiedades. PACELC amplía el análisis y también pregunta qué se prioriza entre latencia y consistencia cuando no hay partición. Ahmed et al. explican ambos marcos para sistemas replicados [6].

La unidad analizada aquí es la *cola de quórum*, no todo RabbitMQ. La documentación oficial indica que estas colas privilegian la consistencia de los datos confirmados: para operar necesitan una mayoría de sus miembros y no consideran seguros los mensajes que el broker aún no confirmó al productor [2]. En el experimento realizado, al detener dos de tres nodos, el miembro restante quedó en estado `pre_vote` y no se observó un líder operativo. Esto concuerda con la dependencia del quórum, aunque debemos dejar claro que los nodos fueron detenidos deliberadamente: **no se simuló una partición de red**.

Con respecto a PACELC, las confirmaciones de publicación y la replicación entre varias réplicas implican coordinación adicional. En términos de diseño se privilegia la seguridad de los datos frente a la mínima latencia en ciertas operaciones. Sin embargo, no registramos tiempos de respuesta comparables, por lo que no asignamos una clasificación PACELC global ni afirmamos cuánto tarda cada operación en RabbitMQ. Esta conclusión es cualitativa y está apoyada en la documentación, no en una medición propia [2], [6].

3.3. Modelos de comunicación

El intercambio observado se puede describir como un esquema *productor–broker–consumidor*. El cliente se conecta a un broker mediante una relación cliente-servidor, pero el clúster funciona mediante comunicación entre nodos. No es una red en la que los clientes Python envíen mensajes directamente entre sí. El efecto práctico es un sistema híbrido: acceso cliente-servidor hacia el broker y coordinación distribuida en el interior de la cola replicada.

Desde el punto de vista de la aplicación, la mensajería es asíncrona porque la publicación no requiere que el consumidor esté conectado al mismo tiempo. En la interfaz web se observó una cola con tres mensajes listos y cero consumidores conectados. Más tarde se recuperaron esos mensajes. Conviene distinguir esta propiedad de las confirmaciones: en el protocolo pueden existir solicitudes y respuestas de control para confirmar una publicación o un consumo. Maharjan et al. describen los intercambios de mensajes y el desacoplamiento de aplicaciones [1].

Entre los mecanismos empleados se encontraron sockets TCP para transportar AMQP, operaciones de publicación y recuperación con Pika y comunicación interna de replicación basada en Raft. Zhang, Kang y Lee explican los estados de líder, seguidor y candidato de Raft y la relevancia de una mayoría para confirmar entradas del registro [7]. Ese artículo estudia una variante llamada FlexRaft; en RabbitMQ el mecanismo específico debe respaldarse con su documentación oficial, no atribuirle automáticamente dicha variante [2].

3.4. Tolerancia a fallos y recuperación

El principal mecanismo de tolerancia a fallos estudiado fue la replicación de la cola de quórum. Con tres miembros, una mayoría equivale a dos. Esto permite soportar la caída de uno sin perder necesariamente la posibilidad de operar. Cuando desaparece la mayoría, el protocolo no puede continuar como si todas las réplicas estuvieran de acuerdo. La documentación oficial explica que los seguidores que regresan pueden ponerse al día con el líder actual [2].

En la prueba local, después de publicar cinco mensajes y recibir las confirmaciones del broker, se detuvo `rabbit1`, líder inicial. Al consultar desde `rabbit2`, se observó que `rabbit3` había asumido el liderazgo; la consulta de colas seguía mostrando cinco mensajes pendientes. Posteriormente, el consumidor conectado a `rabbit2` obtuvo los cinco mensajes y envió confirmaciones manuales. Al reiniciar `rabbit1`, los miembros presentaron el mismo índice de confirmación 51.

Una segunda prueba detuvo `rabbit2` y después `rabbit3`. El nodo `rabbit1` quedó solo y apareció como `pre_vote`. Cuando se reinició `rabbit3`, ambos nodos recuperaron la mayoría y la cola volvió a mostrar cinco mensajes pendientes. Estos fueron consumidos; luego se reinició el tercer nodo y las tres réplicas alcanzaron el índice 70. El mismo índice es evidencia de sincronización del registro en el momento consultado, pero no es una prueba de rendimiento ni de ausencia de todos los fallos posibles.

3.5. Un incidente real documentado

El postmortem de Clever Cloud correspondiente al 2 de agosto de 2024 describe una interrupción amplia de su plataforma causada inicialmente por problemas durante una intervención del sistema de mensajería Apache Pulsar y por la presión posterior sobre la infraestructura. En la cronología se indica que a las 16:05 (CEST) un nodo RabbitMQ del clúster utilizado por el orquestador estaba sobrecargado debido a un hipervisor saturado. A las 16:15, el equipo retiró ese nodo para impedir que siguiera ralentizando la orquestación, tras lo cual el orquestador volvió a funcionar correctamente [8].

Este caso no prueba un defecto del algoritmo Raft ni corresponde al mismo tipo de cola que nuestro ensayo. Su importancia es distinta: incluso si un broker tiene mecanismos de réplica, la disponibilidad real depende de los recursos de la infraestructura, de la supervisión y de la capacidad de aislar nodos problemáticos. En este laboratorio los contenedores también compartían un equipo físico; por ello el postmortem refuerza la necesidad de no confundir tres procesos con tres servidores independientes.

4. Experimentación propia

4.1. Entorno y procedimiento

Las pruebas se realizaron en Windows con Docker Desktop, RabbitMQ 4.2.9-management, Erlang 27.3.4.19, Python 3.11.9 y Pika 1.4.4. Se iniciaron `rabbit1`, `rabbit2` y `rabbit3`; posteriormente se unieron los dos últimos al primero mediante `rabbitmqctl join_cluster`.

Se creó la cola `taind01_mensajes` de tipo `quorum`. Las capturas originales, la configuración y los archivos de salida están organizados en `experimento/`; los comandos completos se explican en `experimento/README.md`.

El procedimiento separó tres tipos de observación: estado de los nodos, resultados de publicar/consumir mensajes y comportamiento de las réplicas durante paradas y reinicios. Se utilizaron `cluster_status`, `quorum_status`, `list_queues`, `docker compose stop/start` y programas Python. En las ejecuciones se registraron fecha y hora, nombre del equipo y salidas de PowerShell.

4.2. Resultados de mensajería

Inicialmente se publicaron tres mensajes desde RabbitMQ Management sin consumidores conectados. La cola mostró `Ready=3`, `Unacked=0`; después de recuperarlos con confirmación automática, ambos contadores fueron cero. Luego se creó un productor Python que publicó cinco mensajes con `confirm_delivery()` y un consumidor que recuperó esos cinco mensajes con `basic_get(auto_ack=False)` y confirmó cada uno mediante `basic_ack()`. Se verificó que la cola terminó vacía. Las confirmaciones de publicación y de consumo representan operaciones distintas: el broker primero asume la responsabilidad del mensaje y el consumidor posteriormente señala que lo procesó [5].

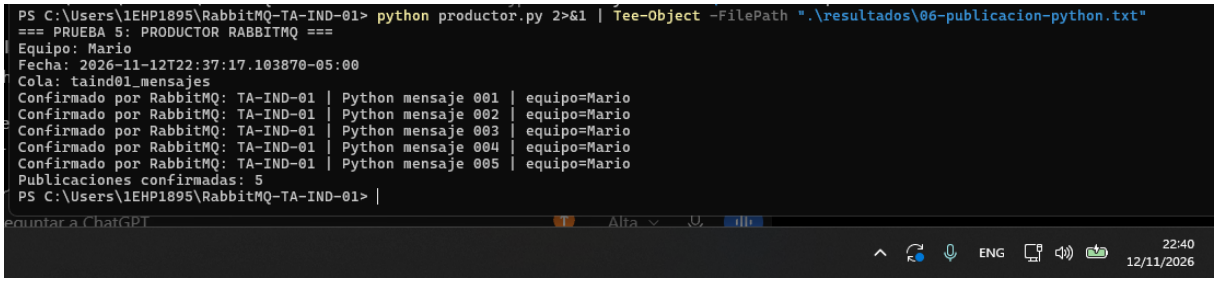


Figura 2: Publicación desde Python: cinco mensajes confirmados por RabbitMQ. Captura original de la terminal del estudiante.

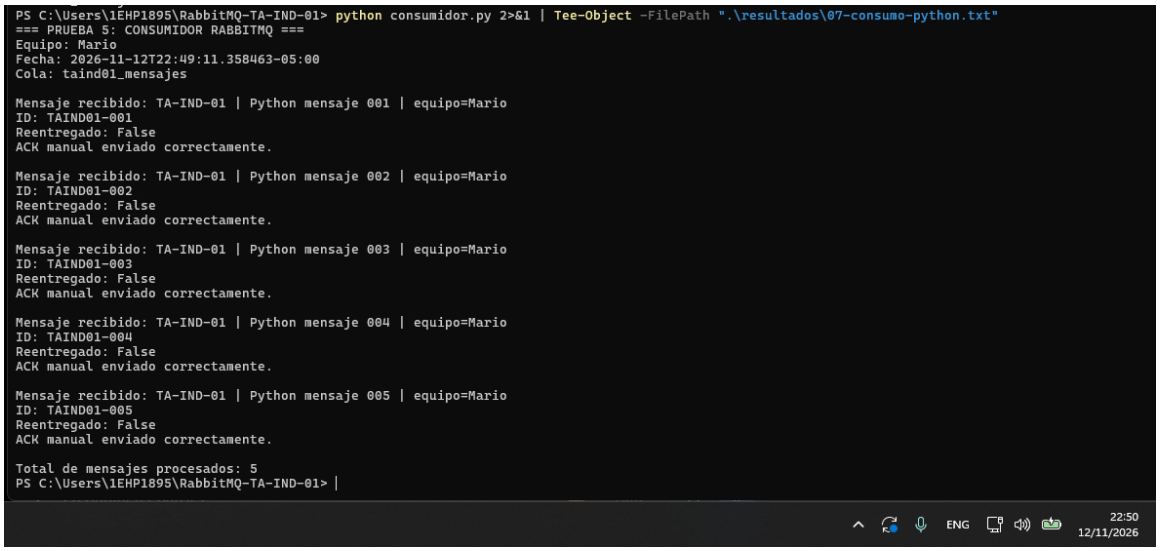


Figura 3: Consumo de cinco mensajes con `basic_ack()` y resultado del programa.

4.3. Resultados de las fallas controladas

La tabla siguiente sintetiza los estados realmente observados, sin introducir tiempos de recuperación que no fueron medidos.

Tabla 2: Resultados observados en las pruebas con la cola de quórum.

Escenario	Miembros activos	Resultado verificado
Clúster inicial	3 de 3	Tres miembros registrados; cola con líder y dos seguidores.
Falla del líder inicial	2 de 3	`rabbit3` asumió liderazgo; cinco mensajes quedaron disponibles y se consumieron.
Reinicio de `rabbit1`	3 de 3	Tres miembros activos, índices de confirmación iguales a 51.
Pérdida de mayoría	1 de 3	`rabbit1` en `pre_vote`; sin líder operativo.
Regreso de `rabbit3`	2 de 3	Recuperó el liderazgo; persistían cinco mensajes pendientes.
Recuperación completa	3 de 3	Mensajes consumidos; tres réplicas con índice de confirmación 70.

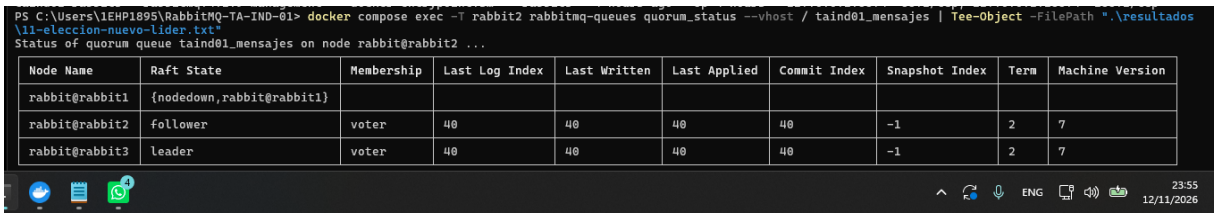


Figura 4: Estado Raft después de detener `rabbit1`: `rabbit3` líder y `rabbit2` seguidor.

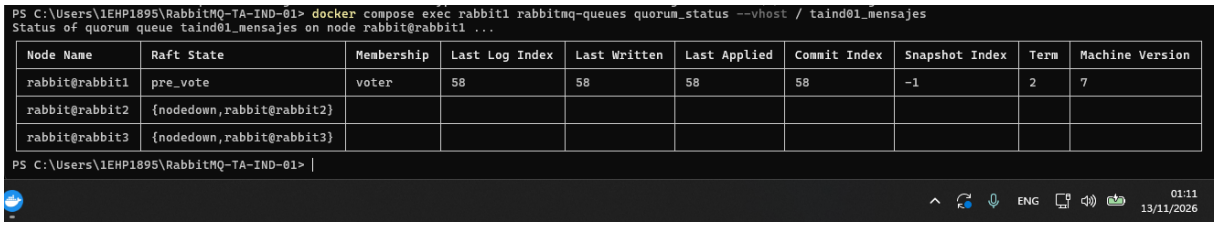


Figura 5: Pérdida de mayoría: `rabbit1` aparece en `pre_vote`; los otros miembros están detenidos.

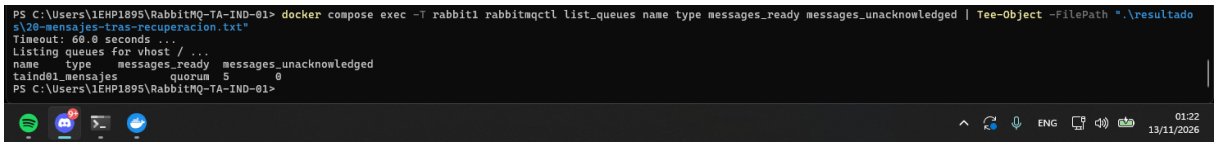


Figura 6: Cinco mensajes pendientes después de restablecer la mayoría con dos miembros.

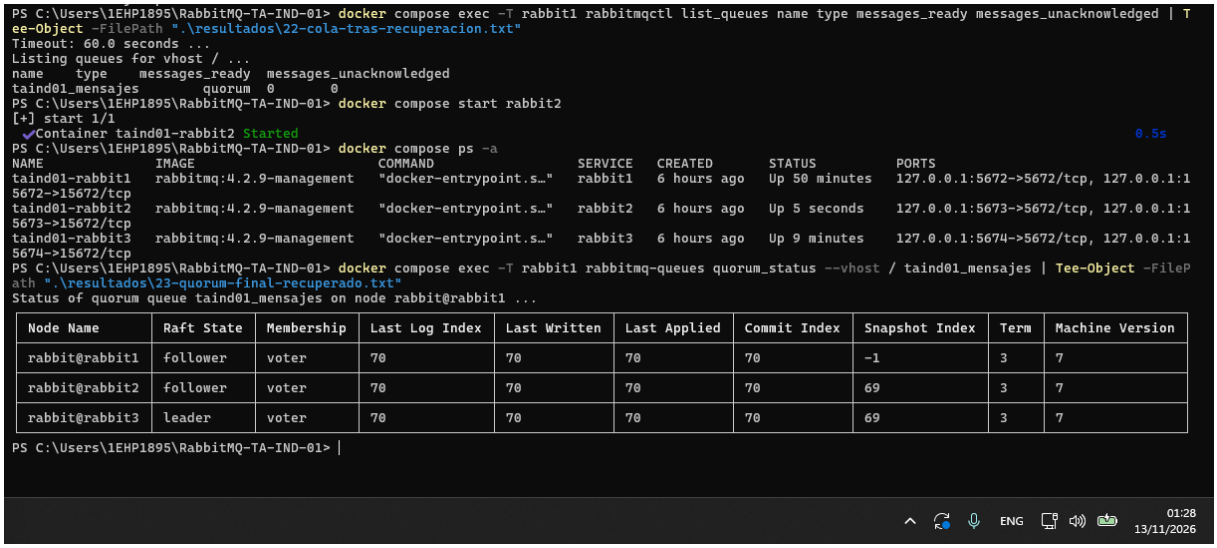


Figura 7: Estado final del clúster: tres réplicas con índice de confirmación 70.

La evidencia permite concluir que los cinco mensajes confirmados se recuperaron tras la caída de un líder y después de restablecer una mayoría previamente perdida. No demuestra que no pueda existir duplicación en reintentos, que la red sea tolerante a cualquier partición o que el mismo resultado se obtenga ante una falla del equipo físico completo. Tampoco se comparó el rendimiento con Kafka mediante un ensayo equivalente.

5. Ejemplos de sistemas del entorno

5.1. Sistema de Gestión Académica de la UTEQ

En la universidad utilizo el SGA para consultar actividades, revisar indicaciones y realizar entregas. Desde el punto de vista del usuario, la plataforma permite que un docente publique una tarea y que el estudiante acceda a ella en otro momento. Este comportamiento recuerda

al desacoplamiento temporal, aunque el SGA no debe confundirse con un broker de mensajes. No tenemos acceso a su arquitectura interna ni se ha comprobado si utiliza colas o réplicas. La relación con la Unidad 1 es la separación entre clientes, servidor y persistencia de la información visible a los usuarios.

5.2. Repositorio compartido en GitHub

Durante esta actividad se utilizaron las funciones web de GitHub para subir archivos, revisar cambios y consultar el historial de commits. La actualización del repositorio es una operación a través de un servicio remoto: no se necesita acceder directamente al disco del servidor que almacena el proyecto. La experiencia ilustra transparencia de acceso desde el punto de vista del usuario. El historial también permite contrastar el orden de las actualizaciones. Esto no implica que se haya examinado la arquitectura interna de GitHub ni sus protocolos de replicación. Ambos ejemplos parten de acciones realizadas en el desarrollo de la materia.

6. Comparación crítica: RabbitMQ y Apache Kafka

Kafka no fue ejecutado en este laboratorio. La comparación se fundamenta en publicaciones científicas que estudiaron ambas tecnologías y debe interpretarse según sus versiones, cargas y entornos de medición. Maharjan et al. comparan brokers en distintos perfiles de carga; Pacella et al. reportan resultados de publicación y consumo en un entorno de manufactura; Kamiński et al. examinan los sistemas desde aplicaciones .NET y Spring Boot [1], [9], [10].

Tabla 3: Comparación de RabbitMQ y Kafka desde la Unidad 1.

Criterio	RabbitMQ	Apache Kafka
Organización del dato	Exchange y colas para enrutar y entregar mensajes.	Temas y particiones con registro de eventos.
Comunicación	Modelo productor–broker–consumidor, comúnmente asíncrono.	Productores y consumidores de flujos de registros.
Persistencia y lectura	Cola y confirmaciones; al confirmar el consumo, la entrega puede retirarse.	Retención configurable del registro; varios consumidores pueden leer según sus posiciones.
Replicación	Cola de quórum con líder y seguidores Raft.	Particiones replicadas con líderes y réplicas.
Tolerancia a fallos	Depende del quórum de cada cola replicada.	Depende de la configuración de replicas y de las particiones.
Compromiso de rendimiento	Las colas de quórum priorizan seguridad de datos y coordinación.	Apropiado para flujos particionados y consumo repetible; rendimiento depende de carga.
Visibilidad del estado	Gestión de nodos, colas, consumidores y confirmaciones.	Gestión de brokers, particiones y desplazamientos del registro.
Escenario preferible	Procesamiento de tareas y entrega entre servicios.	Canalización de eventos, retención de registros y reprocesamiento.

En la tabla 1 del estudio de Pacella et al., para una carga de 10 000 mensajes, Kafka obtuvo 713,25 mensajes/s en el productor y RabbitMQ 626,34; en el consumidor, RabbitMQ alcanzó

2 169,57 mensajes/s frente a 1 895,80 de Kafka [9]. Estos valores pertenecen al entorno publicado por los autores y no deben trasladarse directamente a nuestro laboratorio de cinco mensajes. De manera similar, Kamiński et al. muestran que la comparación puede variar según se utilice Spring Boot o .NET [10]. Por eso no considero correcto declarar un ganador absoluto: primero hay que precisar si importa más la entrega de tareas, la retención de eventos, el reprocesamiento o un volumen alto de mensajes.

7. Postura crítica

La principal fortaleza que observé en RabbitMQ fue la recuperación de mensajes previamente confirmados después de una caída controlada. La cola de quórum conservó un estado coherente con dos miembros y luego recuperó las réplicas detenidas. Esto ofrece una base razonable para aplicaciones en las que perder un mensaje confirmado tendría consecuencias importantes.

Al mismo tiempo, el experimento reveló dos compromisos. El primero es la necesidad de una mayoría: con solo un miembro disponible no hubo líder operativo. La segunda limitación fue el punto de conexión del consumidor; al detener el nodo utilizado inicialmente, fue necesario indicar otro puerto. Ninguno de los dos aspectos anula las ventajas del sistema, pero ambos importan al diseñar una solución que deba continuar funcionando ante fallos reales. La guía de RabbitMQ también indica que las colas de quórum no son apropiadas para todos los casos, especialmente cuando lo más importante es la latencia mínima, hay muchas colas temporales o se requieren patrones de lectura repetida [2].

A diferencia de una instalación con tres máquinas independientes, todos los contenedores de este trabajo compartían la misma computadora. Si el equipo anfitrión se apagara, las tres réplicas quedarían fuera de servicio. Además, no medimos latencias, no ensayamos mensajes sin ACK tras la muerte del consumidor y no hicimos pruebas de particiones reales. Estas limitaciones impiden afirmar que RabbitMQ sea siempre más disponible que Kafka o que mantenga entrega exactamente una vez ante cualquier escenario.

Para sistemas que necesitan mantener eventos durante períodos prolongados y permitir reprocesamiento por múltiples grupos de consumidores, Kafka puede ser una alternativa más natural. En cambio, para distribuir trabajos entre servicios con enrutamiento flexible y confirmaciones, RabbitMQ encaja con el tipo de sistema examinado. Las publicaciones comparativas señalan que el contexto de uso y el entorno de desarrollo condicionan cualquier decisión [1], [10].

8. Conclusiones

El primer resultado es que RabbitMQ permite separar la publicación del consumo. Esto se observó tanto en el panel de administración como en los programas Python: los mensajes podían permanecer pendientes y fueron recuperados posteriormente con confirmaciones. No se trató solo de una definición teórica, sino de un comportamiento visible en las consultas realizadas.

El segundo resultado es el alcance concreto de la tolerancia a fallos de la cola de quórum. Con tres

miembros pudo continuar funcionando después de perder al líder original, y los cinco mensajes confirmados se consumieron desde otro nodo. En cambio, con un único miembro no se observó un líder operativo. La pérdida temporal de mayoría no implicó pérdida definitiva de los cinco mensajes: al iniciar otro miembro fueron recuperados y consumidos.

Finalmente, la replicación por consenso ayuda a conservar mensajes, pero no elimina el costo de coordinar réplicas ni resuelve automáticamente la recuperación de conexiones del cliente. La elección entre RabbitMQ y Kafka debe basarse en la necesidad de enrutamiento, retención, reprocesamiento y rendimiento, no en una clasificación general de mejor o peor. Para ampliar este estudio harían falta pruebas de latencia, particiones de red y procesos concurrentes, además de un despliegue sobre máquinas físicas independientes.

Declaración de uso de inteligencia artificial

Para preparar esta actividad utilicé ChatGPT de OpenAI (GPT-6) como apoyo en la organización de los pasos, la elaboración y corrección de scripts de Python, la interpretación de material en inglés y la revisión de la redacción y del código LaTeX. Contrasté los comandos con las salidas de mi entorno Docker. Me ayudo tambien con los comandos para los .md

Referencias

[1] R. Maharjan, M. S. H. Chy, M. A. Arju y T. Cerny, «Benchmarking Message Queues,» *Telecom*, vol. 4, n.º 2, págs. 298-312, 2023. DOI: 10.3390/telecom4020018

[2] RabbitMQ Team. «Quorum Queues.» Documentación de RabbitMQ 4.2, RabbitMQ, visitado 13 de nov. de 2026. dirección: https://www.rabbitmq.com/docs/4.2/quorum-queues

[3] B. Chalabi e Y. Slimani, «Multi-Version Managers for Large Scalable Data-Management Systems,» *Future Internet*, vol. 18, n.º 7, 2026, **artno** 358. DOI: 10.3390/fi18070358

[4] S. Attar-Khorasani, L. Sherpa, M. Lieber y S. Ghiasvand, «Seamless transitions: a comprehensive review of live migration technologies,» *Journal of Cloud Computing*, vol. 15, 2026, **artno** 42. DOI: 10.1186/s13677-026-00883-9

[5] RabbitMQ Team. «Reliability Guide.» Documentación de RabbitMQ 4.2, RabbitMQ, visitado 13 de nov. de 2026. dirección: https://www.rabbitmq.com/docs/4.2/reliability

[6] J. Ahmed, A. Karpenko, O. Tarasyuk, A. Gorbenko y A. Sheikh-Akbari, «Consistency issue and related trade-offs in distributed replicated systems and databases: a review,» *Radioelectronic and Computer Systems*, n.º 2, págs. 171-179, 2023. DOI: 10.32620/reks.2023.2.14

[7] M. Zhang, Q. Kang y P. P. C. Lee, «FlexRaft: Exploiting Flexible Erasure Coding for Minimum-Cost Consensus and Fast Recovery,» *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, n.º 10, págs. 1826-1840, 2024. DOI: 10.1109/TPDS.2024.3443424

[8] Clever Cloud. «Postmortem: 2024-08-02,» Clever Cloud, visitado 13 de nov. de 2026. dirección: https://www.clever.cloud/developers/postmortem/2024-08-02/

[9] M. Pacella, A. Papa, G. Papadia y E. Fedeli, «A Scalable Framework for Sensor Data Ingestion and Real-Time Processing in Cloud Manufacturing,» *Algorithms*, vol. 18, n.º 1, 2025, **artno** 22. DOI: 10.3390/a18010022

[10] F. Kamiński, R. Kłonica y B. Pańczyk, «Comparative Performance Analysis of RabbitMQ and Kafka Message Queue Systems in Spring Boot and ASP.NET Environments,» *Journal of Computer Sciences Institute*, vol. 37, págs. 457-462, 2025. DOI: 10.35784/jcsi.7977

Anexo A. Bosquejo manuscrito de arquitectura

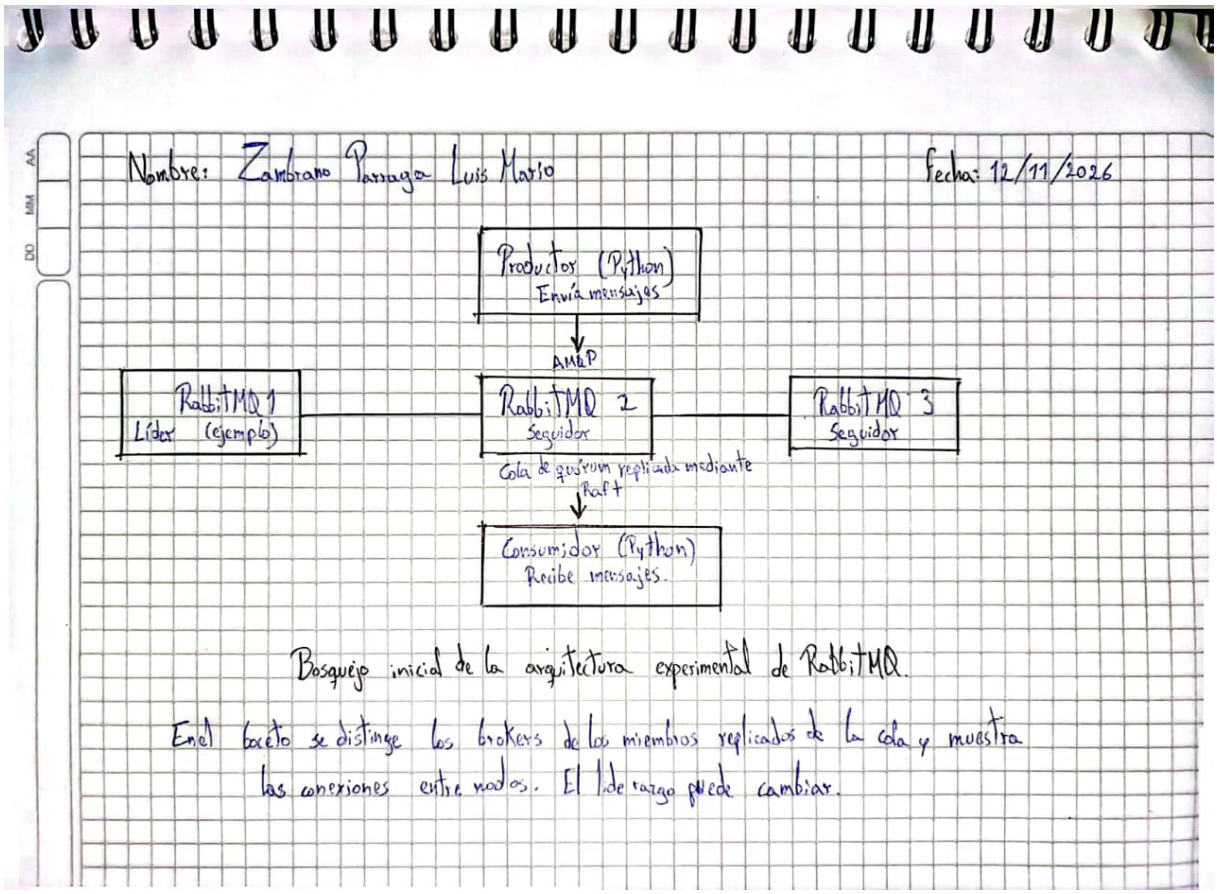


Figura 8: Bosquejo inicial de la arquitectura elaborado a mano, con nombre y fecha visibles en la fotografía original.

Anexo B. Evidencias anotadas de lectura

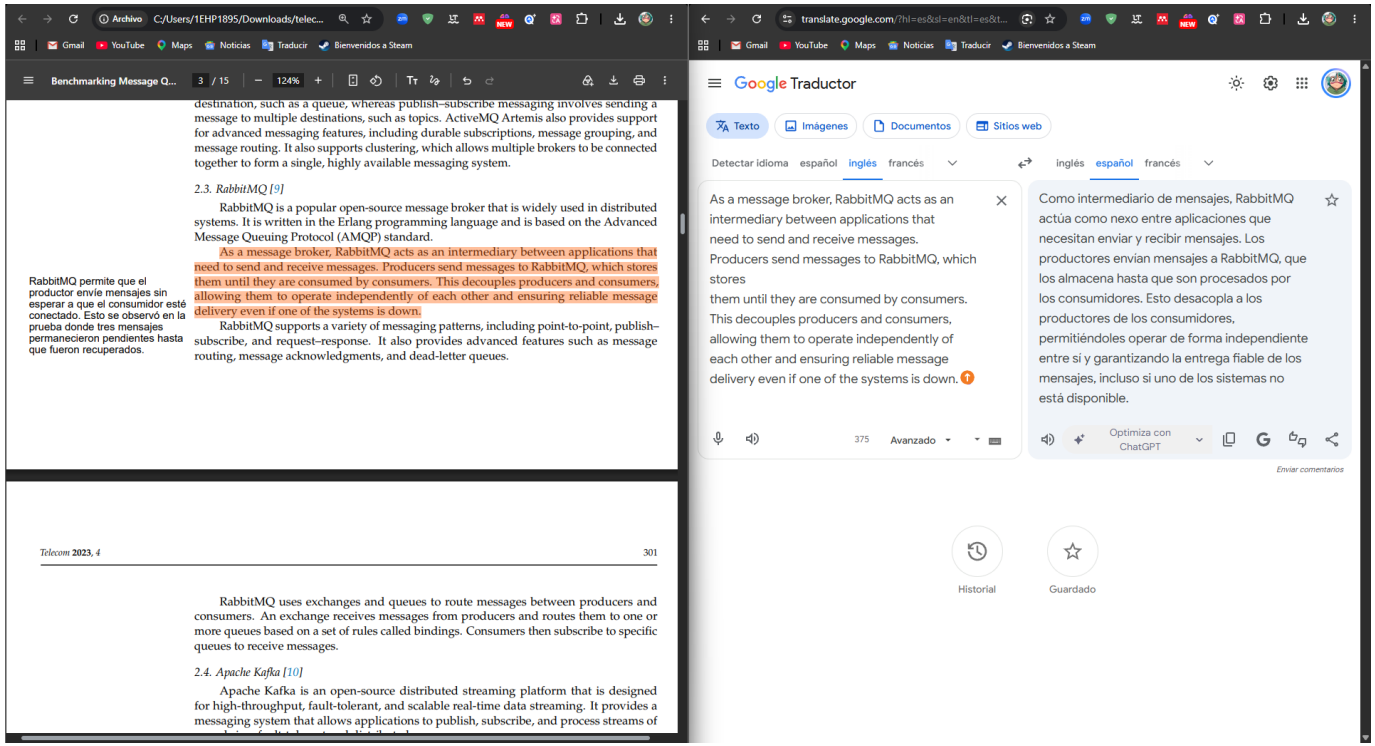


Figura 9: Maharjan et al. (2023), página 300. Desacoplamiento del productor y consumidor [1].

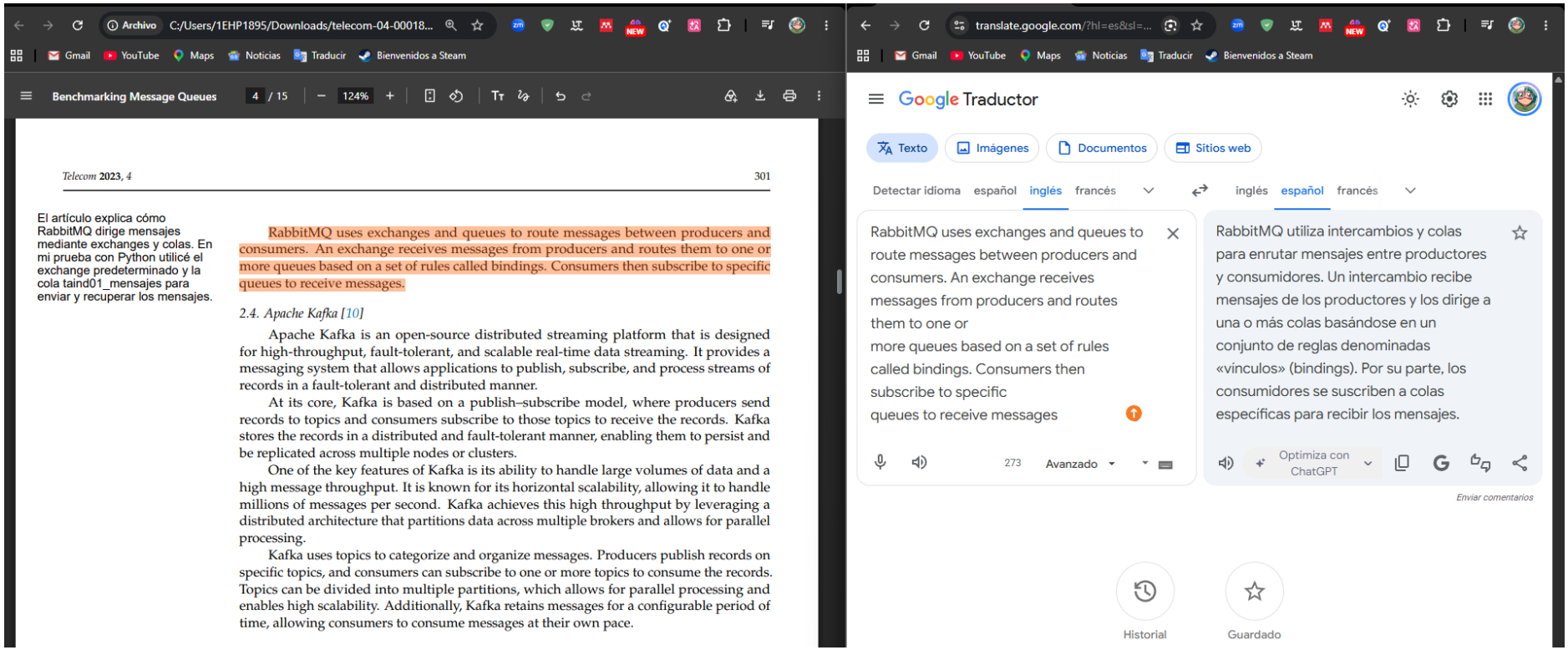


Figura 10: Maharjan et al. (2023), página 301. Exchanges, colas y bindings [1].

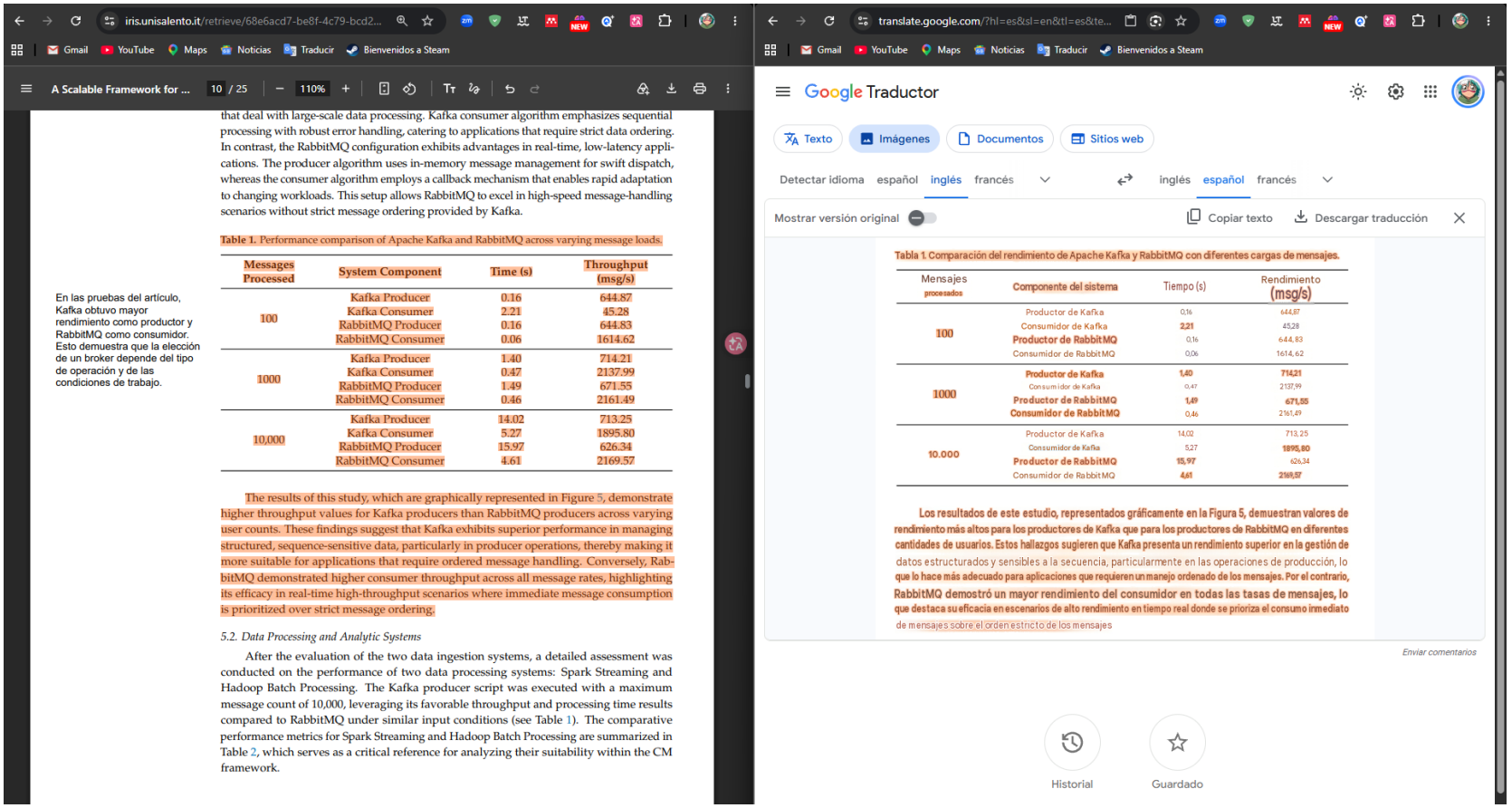


Figura 11: Pacella et al. (2025), página 10 del PDF. Rendimiento comparativo [9].

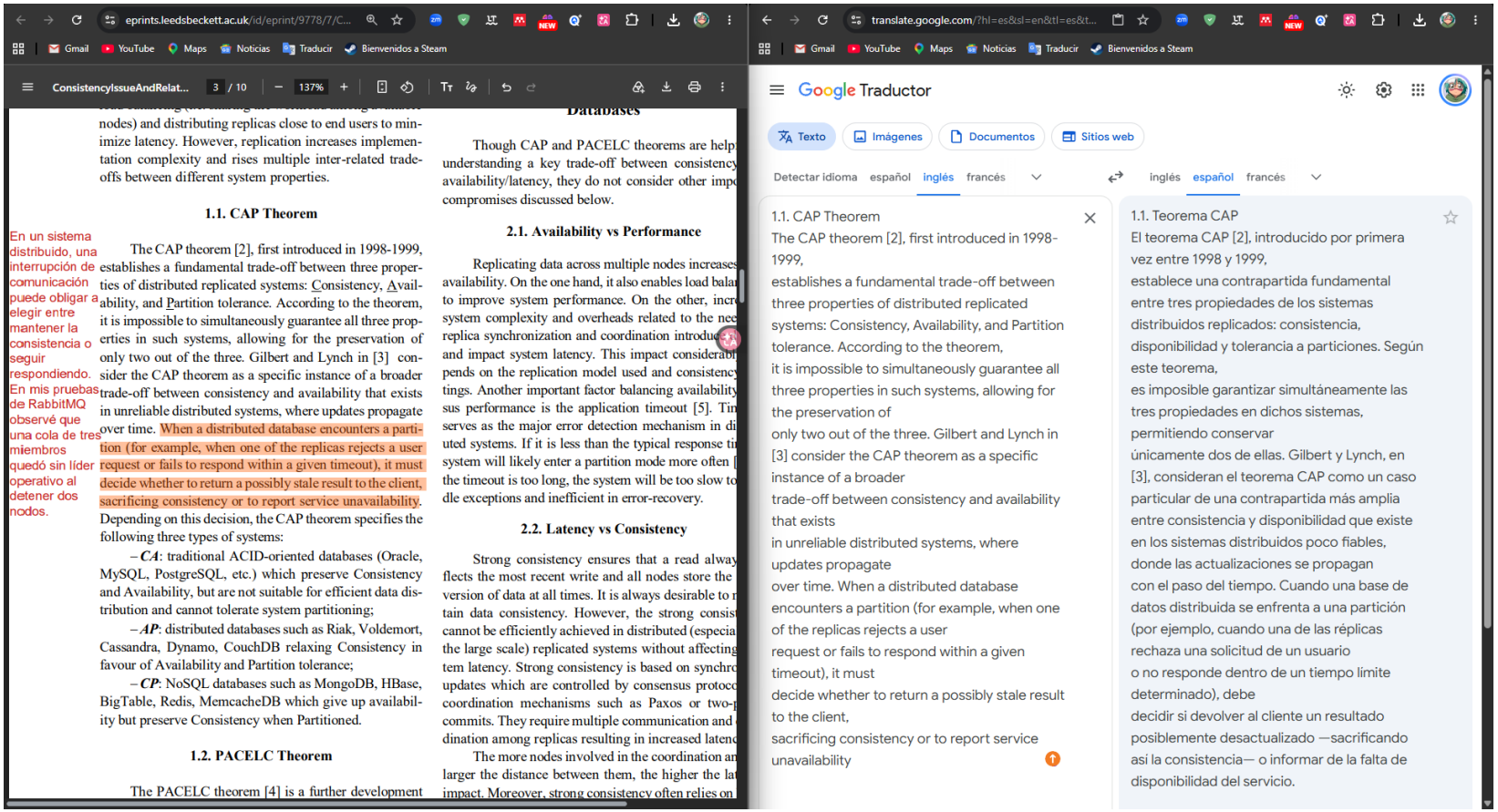


Figura 12: Ahmed et al. (2023), sección 1.1. CAP [6].

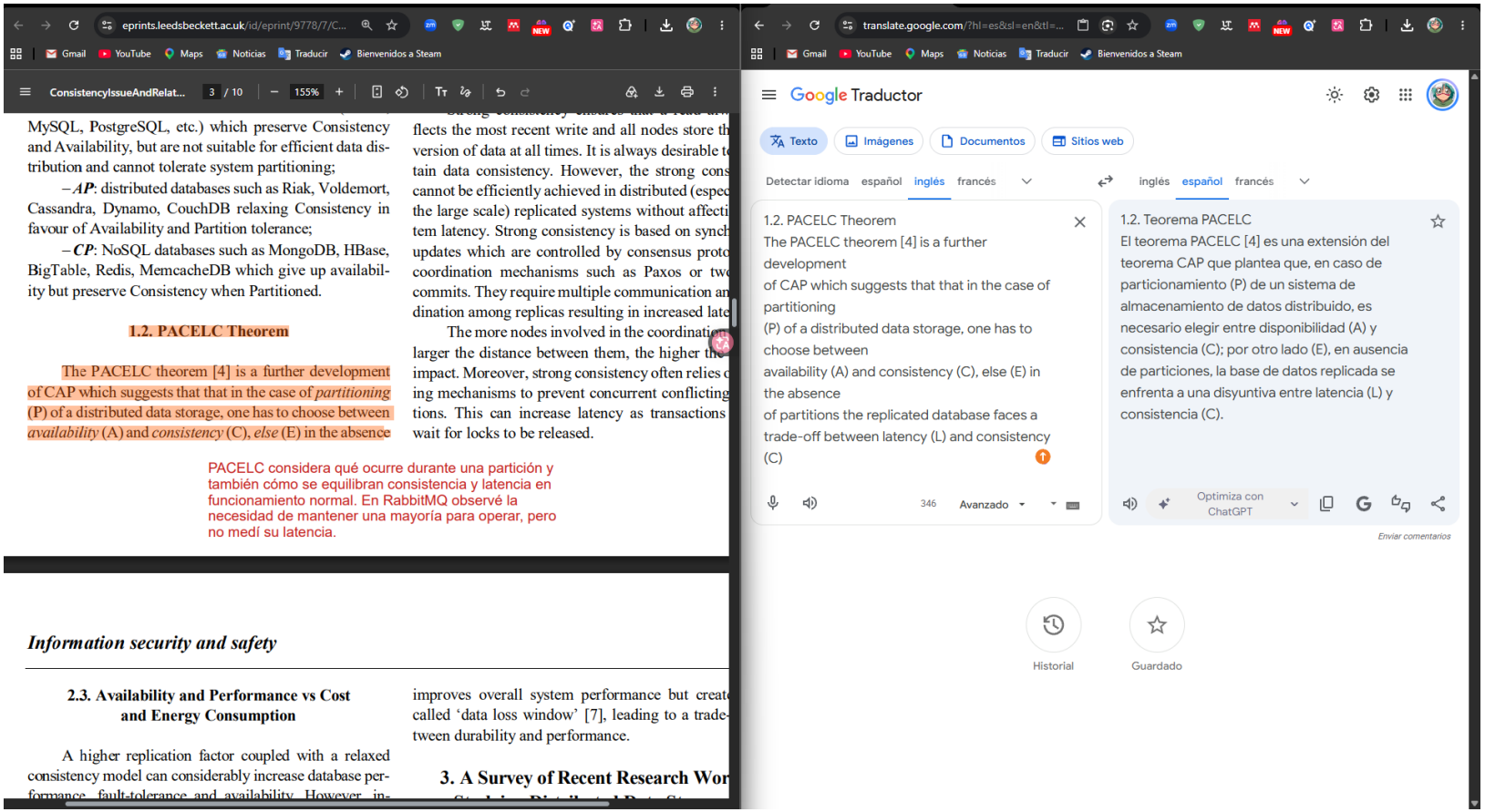


Figura 13: Ahmed et al. (2023), sección 1.2. PACELC [6].

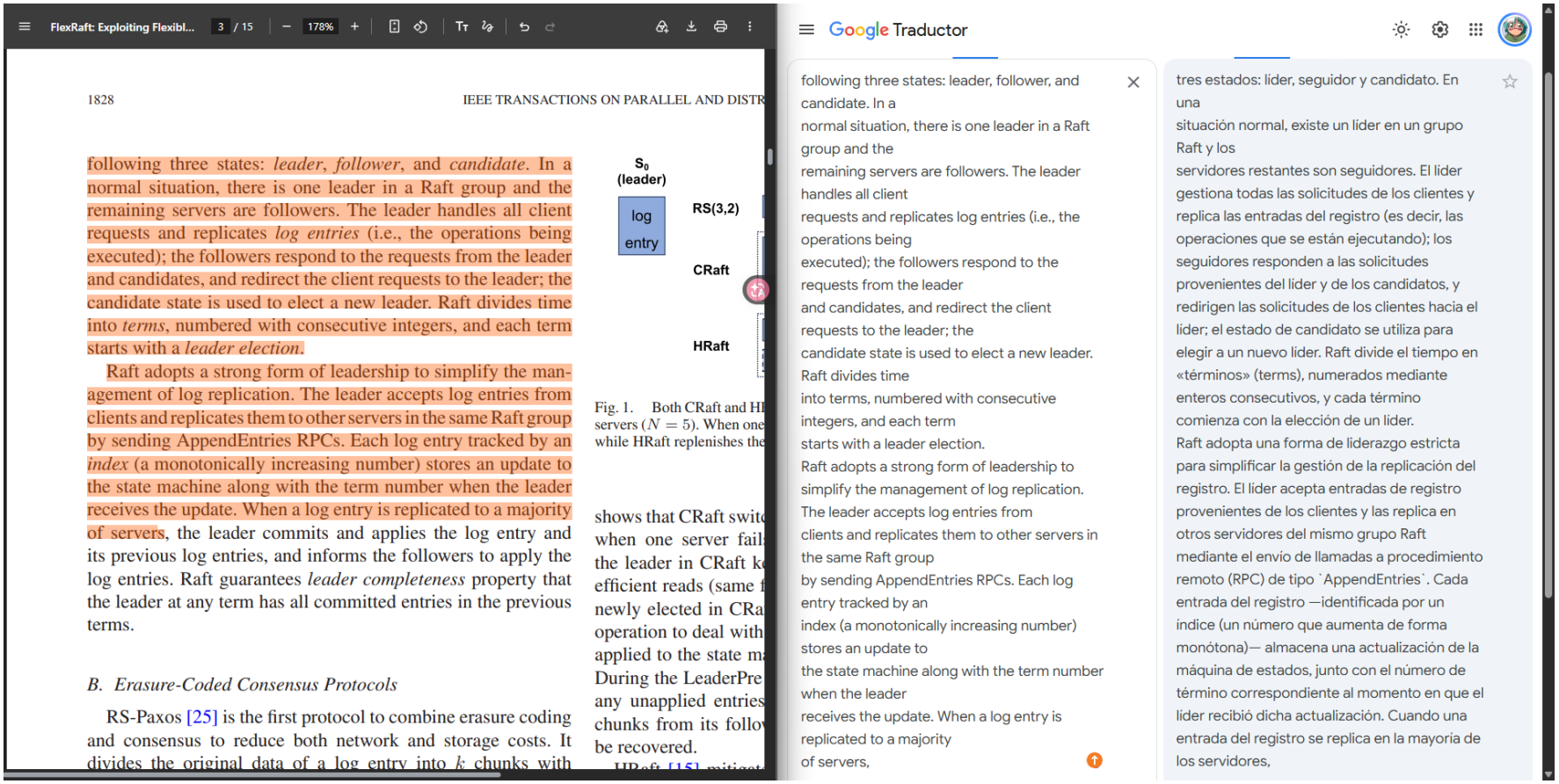


Figura 14: Zhang et al. (2024), página 1828. Raft, líder y mayoría [7].

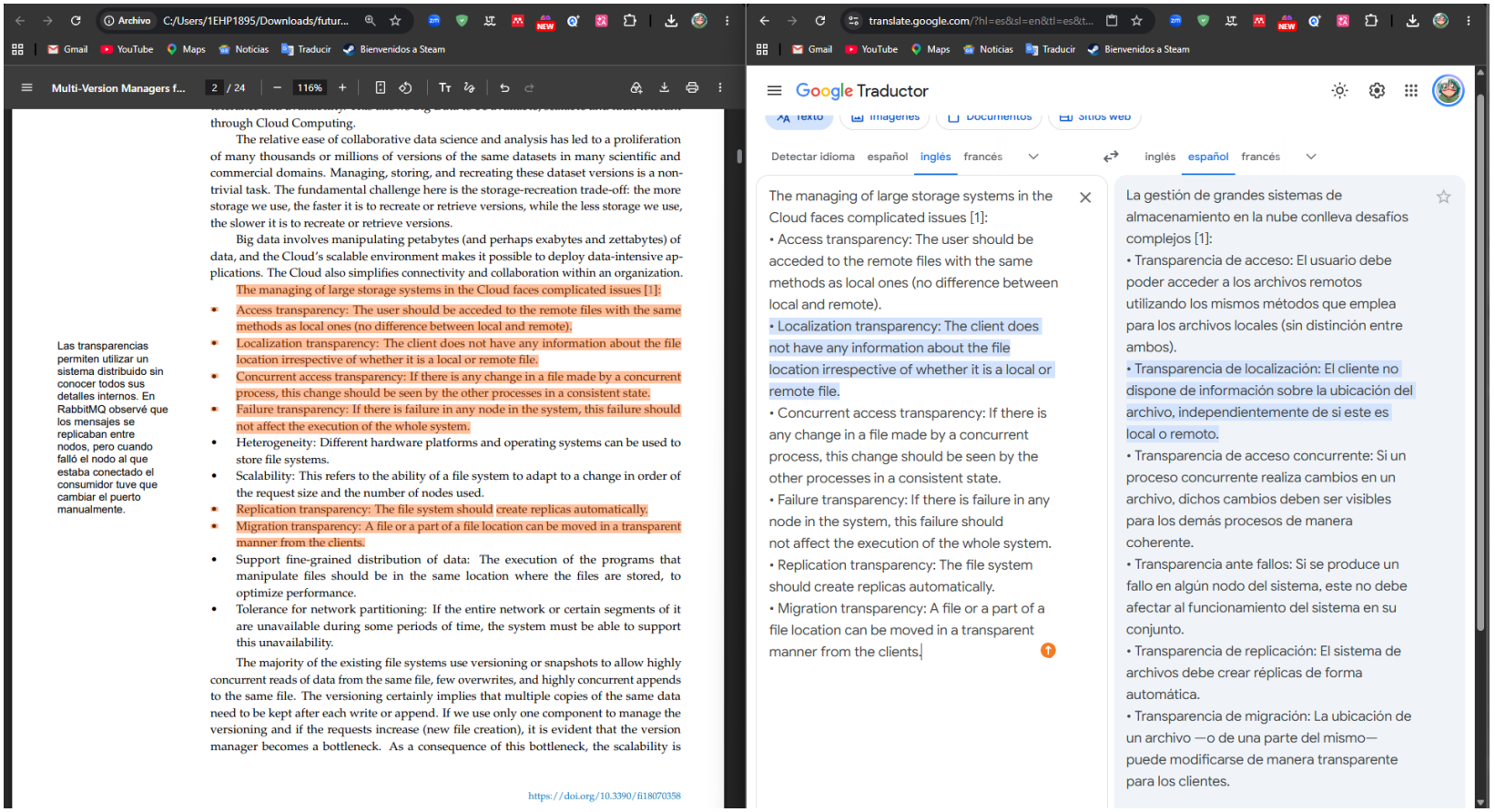


Figura 15: Chalabi y Slimani (2026), introducción. Transparencias de distribución [3].

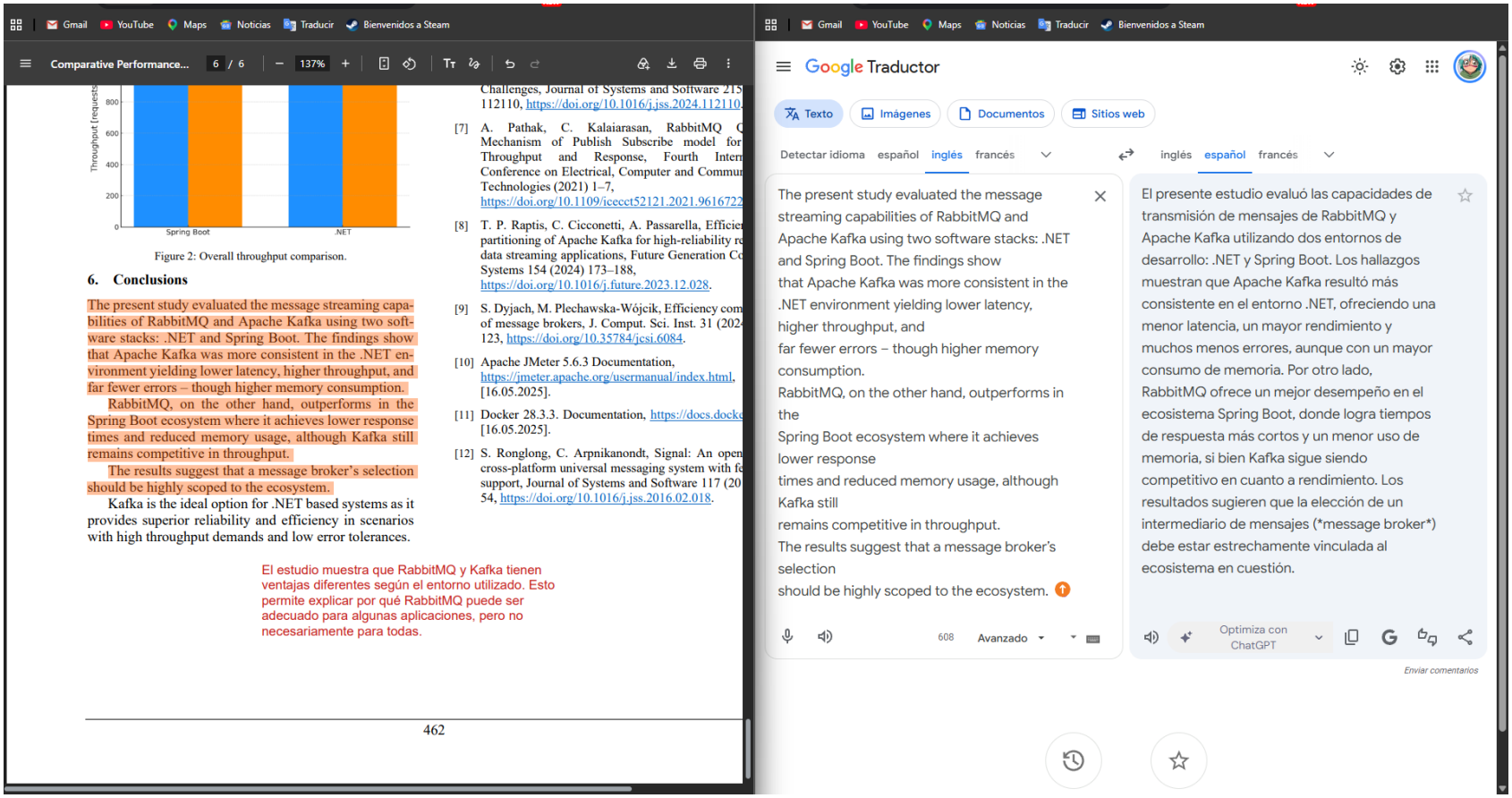


Figura 16: Kamiński et al. (2025), página 462. Compromisos de rendimiento [10].

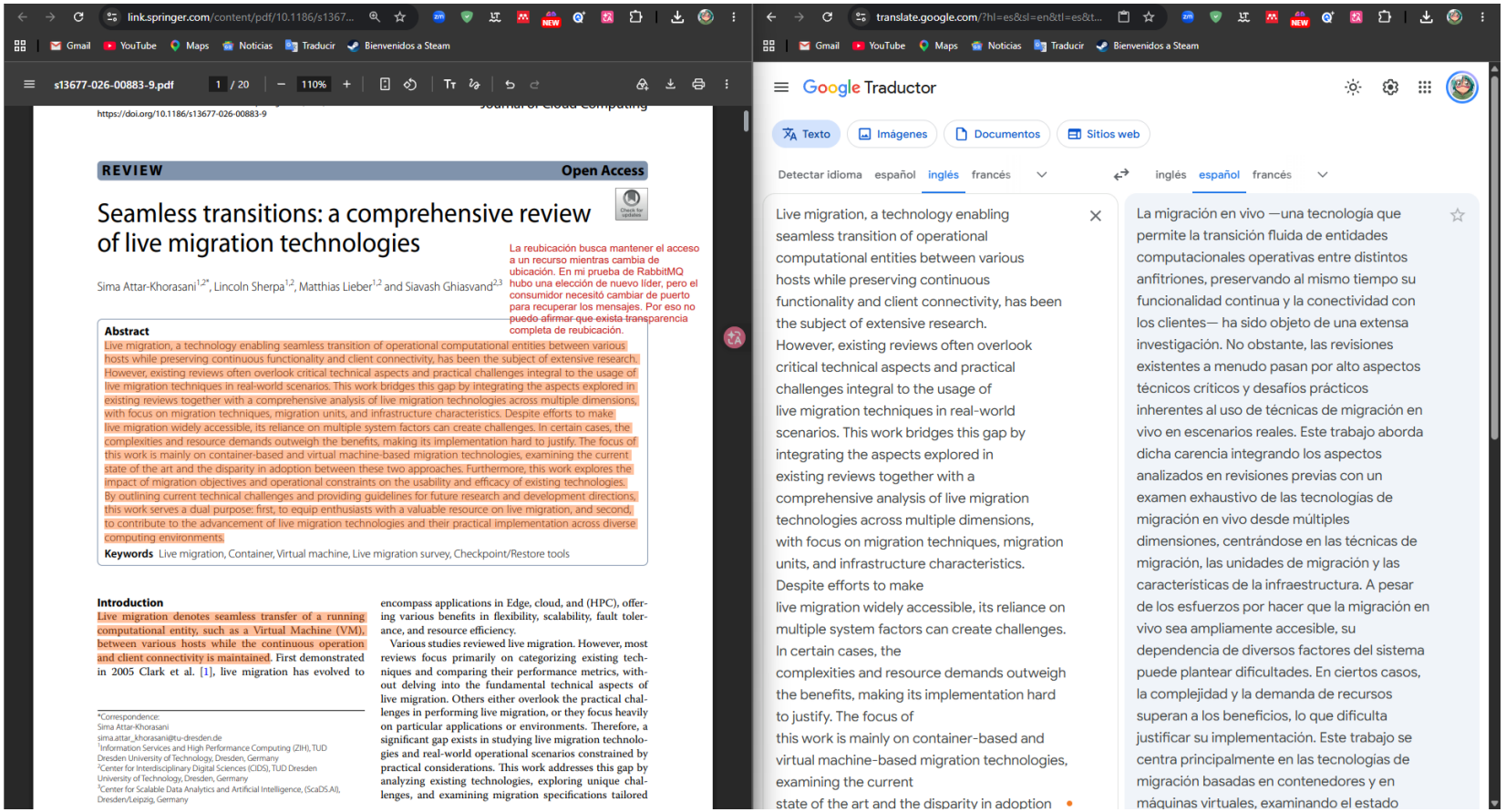


Figura 17: Attar-Khorasani et al. (2026), introducción. Continuidad durante migración en vivo [4].